Putting it together

The Raspberry Pi, the communication network, and every file in your node

EEE40016 Design of Smart Power Grids
Node: digital twin server (Munir Aslam)
12 October 2026, for the integration session

> How to run everything together with the Raspberry Pi, step by step; how the laptop, the broker and the Pi talk to each other; where each teammate's code goes; what to do when something does not connect; and what every file in your node is for. To copy and paste the commands, use **team/README.md**. The street model and the results are explained in **Digital_Twin_Guide.pdf**.

Contents

1. The short version 3

2. How the parts talk to each other 4

3. Step by step 6

4. Where each teammate's code goes 8

5. When something goes wrong 9

6. Every file in your node 10

1. The short version

The four parts run together for the first time, with real messages going between the laptop and the Raspberry Pi. Everything that can be prepared on the laptop is ready and has been rehearsed, and the laptop can drive the whole run by itself.

What	Runs on	Whose
The digital twin: street model, safe-limit search, controllers, experiments	laptop	Munir
The comms injector and trust score	laptop, inside the twin	Hoang
The battery, BMS and EMS	laptop, or the Pi (optional)	Gia
The charge-point controller for all 22 chargers	Raspberry Pi	Lomitha
The MQTT broker, which passes messages between them	the laptop	shared

Gia runs the session on her laptop with her Pi, from the handover zip; its `README_FIRST.md` has the same steps as this guide.

The easy way, in five moves:

1. Put the laptop and the Pi on the same phone hotspot, with SSH switched on on the Pi (Section 3).
2. Find the Pi's address from the laptop: `py tools/pi_session.py --find`.
3. Run `py tools/pi_session.py --pi <user>@<Pi address>` and type the Pi's password when asked. It ends with EQUIVALENT.
4. Add the teammates' parts with `--team`, one at a time, then all together with the battery on the Pi: `--team all --battery-on-pi`.
5. If the Pi will not connect, `py tools/pi_session.py --local` runs the same evening on the laptop alone, for the demo.

Nothing has to be installed on the Pi, and nothing needs allowing through the laptop's firewall. Section 3 has the details, and the by-hand route.

Already tested

- A full rehearsal on the laptop, with the Pi package unzipped into a separate folder and run as its own program standing in for the Pi: one evening over MQTT, EQUIVALENT to the simulator at every step, with replies in 5 to 12 ms.
- The same with Python told to ignore every installed package, which proves the Pi's program needs only the MQTT library it carries with it.
- Lomitha's own code (one gateway per charger, with the IEC 61851 pilot rules), on both sides with `--team lomitha`: EQUIVALENT. Switched to ideal settings it gives the stand-in's answer exactly; with its real settings, 0.5 to 2.4% less energy and no extra violations.
- All three parts with the battery on the Pi (`--team all --battery-on-pi`): EQUIVALENT, battery state of charge included, about 1 ms per battery call.
- The SSH tunnel, with a stand-in for it that accepts the connection before any broker answers, as the real one does: the Pi waits, connects, EQUIVALENT.

- The one-command run with `--local`, and its error messages for a wrong address, no SSH, and a typo in `--team`.
- Pulling the Pi offline: the laptop refuses to start and prints a checklist.
- Each teammate's starting file passes the acceptance tests (V2, V5, V6, V7), and one experiment run on all three files matches the main results exactly.
- None of these changes altered a single result: all golden runs identical, and V9 passed on 11 October, eight days after it was recorded.

Not tested yet, because it needs the real hardware: the Raspberry Pi itself, SSH into it from the laptop, and the tunnel over a real network. The commands are written for all three, but the first run with the Pi is the first time they meet.

2. How the parts talk to each other

Everything goes through an **MQTT broker**, a small program that receives each message and passes it on to whoever has asked for that kind of message (that is, has subscribed to that topic). The laptop and the Pi never connect to each other directly; both connect to the broker.

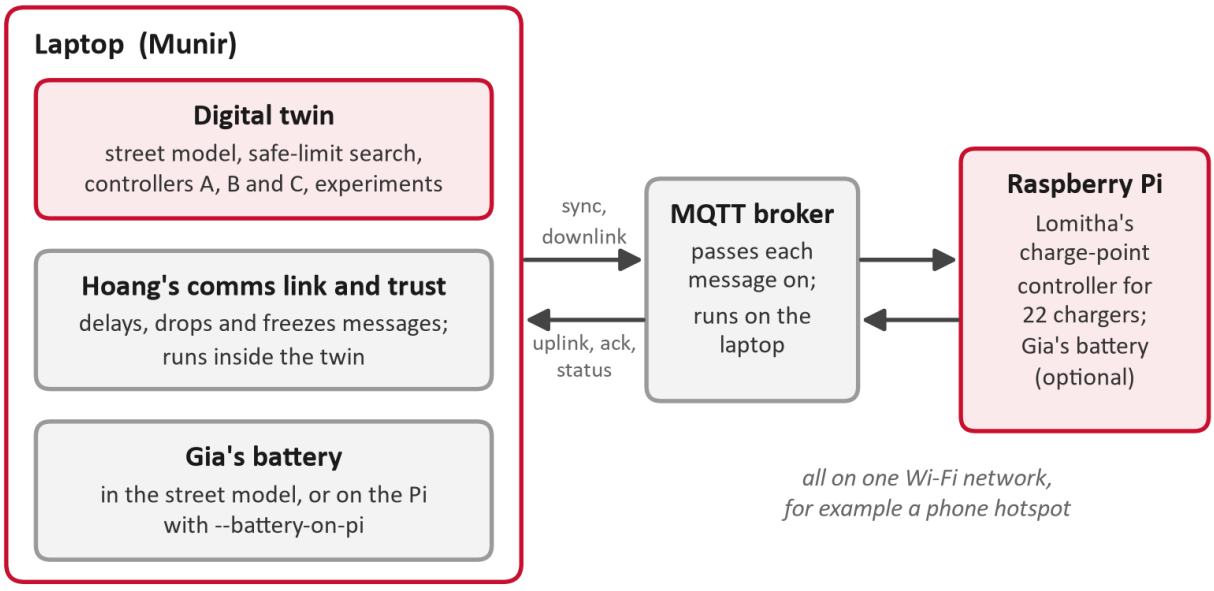


Figure 1. What runs where. Hoang's code runs inside the twin on the laptop; the charge-point controller is on the Pi, and Gia's battery can be either.

How the Pi reaches the broker

By default the Pi reaches the broker through the SSH connection the laptop opens to start it (an SSH tunnel): the Pi connects to its own end of the tunnel, and SSH carries the messages to the broker on the laptop. Windows never sees an incoming connection, so nothing needs allowing through the firewall, and the messages travel encrypted. With `--direct` the Pi connects straight to the broker across the Wi-Fi instead, as plain MQTT on port 1883; Windows then asks to allow Python the first time. Either way the messages are the same and so are the results.

The topics

Topic	From → to	What it carries	How often
`eee40016/sync`	laptop → Pi	opens a step; what each house's meter reads	every step
`eee40016/uplink/edge`	Pi → laptop	meter readings and every car's status	10 per step
`eee40016/downlink/edge`	laptop → Pi	the charging limit, its schedule entry and when it is valid	every step
`eee40016/ack/edge`	Pi → laptop	what each charger actually drew	every step
`eee40016/status/edge`	Pi → laptop	"online" or "offline", kept by the broker so the laptop can check the Pi is there	on connect and disconnect
`eee40016/breq/battery`	laptop → Pi	battery on the Pi only: a call on the battery (reset, response, commit)	up to 7 per step
`eee40016/bresp/battery`	Pi → laptop	battery on the Pi only: the answer	one per call

Every message is a small piece of JSON defined in `src/messages.py`. That file is the contract: the laptop and the Pi must have the same version (version 2), or messages are rejected rather than half-read. The Pi package always carries the right copy.

One step, in order

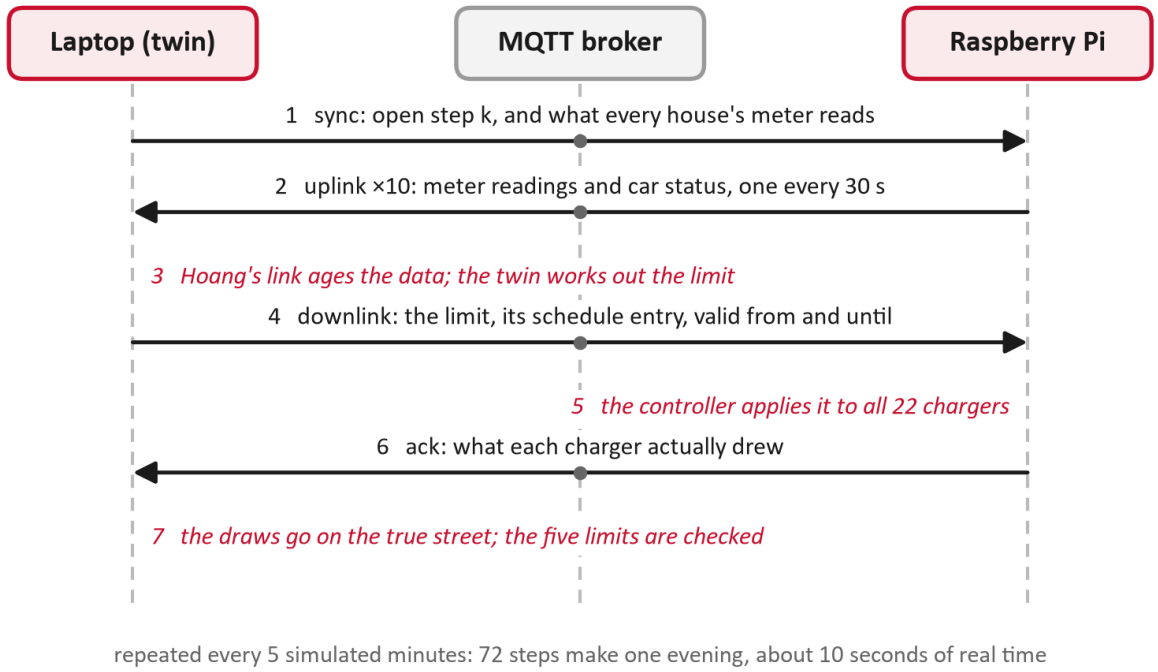


Figure 2. The messages in one 5-minute step. The laptop waits for the Pi's reply before moving on, so the two stay in step.

Two kinds of delay, kept apart

The study's delay and packet loss (for example 60 seconds and 10%) are **simulated** inside the twin by Hoang's comms link, in simulated time. The real network between the laptop and the Pi adds only a few milliseconds. That real delay is measured and reported, but never added to the study. The Pi runs on the simulated clock carried in each message, not its own clock. This is why a whole evening takes about ten seconds, and why the Pi run can match the simulator exactly.

> The electricity network itself (the street, its cables and transformer) is explained in **Digital_Twin_Guide.pdf**, section 3. This guide is about the communication network between the devices.

3. Step by step

Before you start

- **On the Pi, once** (the only part that needs a keyboard and screen on it): switch SSH on with `sudo raspi-config`, Interface Options, SSH (or tick it in Raspberry Pi Imager when making the SD card), and join the phone hotspot.
- Have a phone hotspot ready. University Wi-Fi (eduroam) usually blocks devices from talking to each other.
- Nothing to install on the Pi: the Pi package carries its own copy of the MQTT library, so the Python 3 that comes with Raspberry Pi OS is enough. A laptop new to the project needs one command, once: `py -m pip install -r requirements_laptop.txt`. SSH is built into Windows 10 and 11 and macOS.

To join the hotspot from the Pi's terminal (recent Raspberry Pi OS):

```
sudo nmcli dev wifi connect "<hotspot name>" password "<password>"
```

The easy way: one command from the laptop

With the laptop and the Pi on the hotspot, find the Pi's address from the laptop:

```
py tools/pi_session.py --find
```

It lists the devices on the hotspot that answer SSH; a Raspberry Pi says Debian or Raspbian. Use it on the hotspot, not on university Wi-Fi.

Then run an evening:

```
py tools/pi_session.py --pi <user>@<Pi address>
```

The user is the one made when the Pi was set up. The command rebuilds the Pi package from the laptop's files, copies it to the Pi, starts the broker on the laptop and the Pi's program over SSH, runs the evening, compares it with the simulator and stops the Pi's program again. Type the Pi's password when asked: once to copy, once to start. The Pi's lines start with `Pi  |`. With the battery on the Pi too, the Pi also prints one battery line per step.

Everything at once, with every teammate's part and the battery on the Pi:

```
py tools/pi_session.py --pi <user>@<Pi address> --team all --battery-on-pi
```

Option	Meaning	Default
`--controller`	A fixed, B trust-weighted, C binary fallback	B
`--latency`	simulated delay, seconds, both directions	60
`--loss`	simulated packet loss, 0 to 1	0.10
`--seed`	which simulated evening (7 to 19); both sides get it	7
`--plugin-model`	s1 main case, s2 stress case; both sides get it	s1
`--team`	teammates' files instead of the stand-ins (Section 4)	none
`--static-kw`	the fixed limit for A and C; left out, it is calibrated (about a minute; 2.5 for seed 7)	none
`--no-copy`	the Pi already has the current package: skip the copy and one password	off
`--no-compare`	skip the check against the simulator, for a shorter demo	off
`--battery-on-pi`	run the battery on the Pi as well (Gia's class with `--team gia` or `all`)	off
`--direct`	plain MQTT across the Wi-Fi instead of the SSH tunnel (Windows asks to allow Python)	off
`--dry-run`	print what would run, run nothing	off

Plan B. If the Pi will not connect, the same evening runs with the Pi's program on the laptop, with the same output and the same check:

```
py tools/pi_session.py --local
```

By hand, if you prefer

The same run as separate steps. Find the addresses with `ipconfig` on the laptop and `hostname -I` on the Pi.

1. Start the broker on the laptop with `py tools/broker.py` and leave it open (allow Python through the firewall when asked).
2. Copy `pi_package.zip` to the Pi (USB stick, or `scp pi_package.zip <user>@<Pi address>:`) and unzip it there.
3. On the Pi, in `pi_package`: `python3 run_pi_node.py --broker <laptop address>`. It says "connected". Add `--battery` to run the battery there too (and `--battery-on-pi` on the laptop).
4. On the laptop: `py tools/run_with_pi.py --broker 127.0.0.1 --compare`. Use the same `--seed`, scenario and `--team lomitha` on both sides.

To run the broker on the Pi instead (then nothing on the laptop needs allowing through the firewall, and both sides use the Pi's address):

```
sudo apt install mosquitto
printf 'listener 1883\nallow_anonymous true\n' | \
    sudo tee /etc/mosquitto/conf.d/eee40016.conf
sudo systemctl restart mosquitto
```

Swap in each teammate's code

One person at a time, so a problem is easy to trace: Lomitha first, then Hoang and Gia. Add `--team lomitha` (then `hoang`, `gia`, `all`) to the command and run again. Section 4 has the details.

The demo

A good pair to show, where <pi> is <user>@<Pi address>: B at 60 s and 10% loss (live control working), then C at 300 s and 20% (where the all-or-nothing approach struggles):

```
py tools/pi_session.py --pi <pi> --controller B --latency 60 --loss 0.1
py tools/pi_session.py --pi <pi> --controller C --latency 300 --loss 0.2 --no-copy
```

4. Where each teammate's code goes

Each teammate has a starting file in `team/`. It already works as it is, because it inherits everything from the stand-in. They replace the methods with their own, one at a time, and run their test after each change. They keep the class name and the constructor arguments, and keep it plain Python with no networking inside.

Who	Starting file	Runs on	Plug it in with	Must pass
Hoang	`team/hoang_link.py`	laptop	`--team hoang`	V5, V6
Lomitha	`team/lomitha/` (Lomitha's code) and `team/lomitha_fleet.py`	the Pi	`--team lomitha`	EQUIVALENT
Gia	`team/gia_battery.py`	laptop, or the Pi	`--team gia`; on the Pi add `--battery-on-pi`	V2, V7

`--team` also takes a list, such as `--team hoang,gia`, or `--team all`. Each person checks their own part with the acceptance tests before joining:

```
py tests/test_node_acceptance.py --team hoang
py tests/test_node_acceptance.py --team gia
```

To bring their code in, a teammate copies their one file into `team/` on your laptop. `pi_session.py` copies Lomitha's file to the Pi for you on every run, so edit it on the laptop, not on the Pi. Nothing else changes. The rules each part must follow (R1 to R7) are in `docs/INTERFACES.md` and at the top of each starting file.

> **What EQUIVALENT proves for Lomitha.** It shows Lomitha's controller gives the same answer over MQTT on the Pi as inside the simulator, so the messaging is right. Whether the controller follows the rules is checked by R1 to R5 and by the experiments themselves, which run the same class directly.

The final runs with everyone's code

When all three parts pass, rerun the three experiments with the real code, one after the other. They take about three hours in all, S1 most of it, so keep the laptop plugged in, and in a second window run `py tools/hold_awake.py` so Windows does not go to sleep mid-run.

```
py tools/run_parallel.py --arm "team:S1:results/stages_team:--team all"
py tools/run_parallel.py --arm "team:S2:results/stages_team:--team all"
py tools/run_parallel.py --arm "team:S3:results/stages_team:--team all"
```

Then put the summary together:

```
py src/stages.py --summarise --team all
```

The results go to `results/stages_team`, so they can never overwrite the stand-in results, and each file records which classes made it.

5. When something goes wrong

What you see	Most likely cause	What to do
"REMOTE HOST IDENTIFICATION HAS CHANGED"	the Pi was set up again with the same address	run ssh-keygen -R <Pi address> once, then again
"remote port forwarding failed"	something on the Pi uses the tunnel's port	add --tunnel-port 18831
"the battery on the Pi did not report in"	the Pi program was started without --battery	use pi_session.py with --battery-on-pi, which starts it right
ssh or scp: "Connection refused"	SSH is switched off on the Pi	once, on the Pi: sudo raspi-config, Interface Options, SSH
"Connection timed out" or "cannot find" the Pi	the Pi is not on the hotspot, or the address is wrong	py tools/pi_session.py --find; check the Pi joined the hotspot
"Permission denied"	wrong user name or password	the user and password made when the Pi was set up
"The Pi's program never reached the broker on this laptop"	only with --direct: the Windows firewall, often because the hotspot counts as Public	leave out --direct (the tunnel needs no firewall changes), or allow Python for Private and Public
"already using port 1883"	an old broker or run is still open	close it, then run again
Laptop: "the Pi did not report in"	the Pi program is not running or not connected	look at the Pi | lines; same network
"schema version mismatch"	an old messages.py on one side	run with pi_session.py, which copies a fresh package
NOT EQUIVALENT	different seed, scenario or controller on the two sides	pi_session.py keeps them the same; by hand, match --seed, scenario and --team
"edge sent 3 of 10 samples"	the Pi program crashed or stalled mid-run	read the error in the Pi | lines, fix, run again
"no teammate called ..."	a typo after --team	use hoang, gia, lomitha or all
"no sessions for seed"	a seed outside 7 to 19	use 7 to 19
Ping from the Pi to the laptop fails	Windows ignores pings by default	ping from the laptop to the Pi instead

6. Every file in your node

Grouped by folder, inside `Downloads\EEE40016\project`. The three **stand-ins** are working placeholders for teammates' parts, to be replaced by their own code.

Settings	What it is
`config/study.json`	every setting of the study: time window, limits, margins, controllers, demand case
`config/cable_ratings.json`	cable ratings (provisional) and the transformer's thermal data

Data	What it is
`data/feeder/`	the IEEE European LV Test Feeder package and its published solutions
`data/sessions/`	charging sessions per seed and demand case, frozen so every run uses the same ones
`data/vista/`	the VISTA travel survey and the arrival and energy fit taken from it
`data/pv_profile.csv, ambient.csv, weather/`	solar output per minute and Bureau of Meteorology temperatures

The model (src/)	What it is
`sweep.py`	one simulated evening, step by step: the heart of the study; also calibrates the fixed limit
`envelope.py`	the safe-limit search (bisection on 0.1 kW steps); picks which houses have EVs
`network_state.py`	the five network checks and the transformer temperature model
`solver.py`	the street in OpenDSS (and SINCAL for cross-checks)
`feeder_csv.py`	reads the IEEE feeder package
`controllers.py`	controllers A, B and C
`ev_sessions.py, vista_sessions.py`	who plugs in, when, and how much energy they want
`stages.py`	runs and summarises the experiments S1, S2, S3
`messages.py`	the shared message contract (version 2); everyone uses this exact file
`mqtt_edge.py`	MQTT: the Pi side and the laptop side
`plugins.py`	drops a teammate's class in by name; the --team option
`telemetry.py`	STAND-IN for Hoang: comms injector and trust score
`edge.py`	STAND-IN for Lomitha: the charge-point controller
`battery.py`	STAND-IN for Gia: the battery and its voltage response
`timeout_sensitivity.py, refine_crossover.py`	extra analyses: the timeout test, crossover refinement
`build_sincal_model.py, inspect_sincal_schema.py`	write the street into SINCAL; read SINCAL's database layout

The model (src/)	What it is
`make_scenario_data.py, plot_*.py`	build the solar and weather files; draw the figures

Tests (tests/)	What it is
`test_lomitha_adapter.py`	Lomitha's controller keeps the interface, and what its charging rules change
`v1_benchmark.py ... v9_reproducibility.py`	the validation checks V1, V3, V4, V8, V9
`test_node_acceptance.py`	V2, V5, V6, V7: the teammates' hand-over tests
`test_mqtt_equivalence.py`	the MQTT path gives the simulator's answer
`test_setpoint_schedule.py, golden_runs.py`	the schedule fix; proof that a change did not alter results
`check_model.py, solve_timing.py`	the feeder model's structure; speed

Tools (tools/)	What it is
`pi_session.py`	one command for a whole run with the Pi, or on the laptop alone (--local); --find finds the Pi
`run_with_pi.py`	the laptop side of a run with the Pi
`pi/run_pi_node.py`	the Pi side (copied into the Pi package); --battery serves the battery too
`make_handover.py`	builds the zip for a teammate, with README_FIRST.md
`broker.py`	an MQTT broker on the laptop
`make_pi_package.py`	builds pi_package/ and pi_package.zip, with its own copy of the MQTT library
`run_parallel.py`	the experiment queue, many runs in parallel
`compare_paired.py, compare_sensitivity.py`	before-and-after checks of the schedule fix
`calibrate_preview.py, demand_scenarios.py`	fixed limit per seed; the demand screening
`hold_awake.py`	keeps Windows awake during long runs
`sincal_*.py, avd_*.py, make_avd_package.py`	the SINCAL cross-check, here and on the uni computers

Teammates (team/)	What it is
`hoang_link.py, gia_battery.py`	Hoang's and Gia's starting files (still the stand-ins)
`lomitha/, lomitha_fleet.py`	Lomitha's own code, and the adapter that connects it to the twin
`hoang_prototype/`	Hoang's MQTT chat prototype, with notes
`README.md`	the commands for tomorrow, to copy and paste

Documents	What it is
docs/INTERFACES.md	what each teammate must build (rules R1 to R7)
docs/Digital_Twin_Guide.pdf	the models and the results, explained
docs/Putting_It_Together.pdf	this guide
docs/Your_Files_Explained.pdf	every file here, explained one by one
docs/guide/	the scripts that build the three guides
notes/AUDIT.md	every plan commitment checked, with the final results
notes/plan_amendments.md	all 28 design changes and faults, with reasons
notes/*.md (others)	the result notes for each check and the SINCAL notes
README.md	the project's front page

Results (results/)	What it is
stages/, stress_s2/	the main-case and stress-case results and figures
paired_main/, paired_stress/, mechanism_standin/, sensitivity/	the schedule fix, before and after
timeout_sensitivity/	the timeout test
hil/	runs with the Raspberry Pi; the laptop rehearsals of 11 October are in hil/rehearsal/
golden/, v9/	reference runs for the checks
superseded/	old results; never quote these

Packages	What it is
pi_package/, pi_package.zip	everything that goes on the Pi
avd_crosscheck.zip	the SINCAL check for the uni computers
models/	SINCAL projects
logs/, LOG/, Log.db	run logs and SINCAL's own log files